

SIMD Bitonic Sort

High-Performance Sorting Using SIMD Parallelism

Parallel & Distributed Computing

SIMD • Bitonic Sort • AVX / NEON • Merge Sort

2026

Introduction

Problem Area & Why It Matters

The Problem

- ▶ **Sorting is everywhere**

Graphics, databases, physics engines, compression — all depend on fast sorting.

- ▶ **std::sort is too slow**

Sequential comparison-based sort processes one element pair at a time.

- ▶ **Small arrays are neglected**

Existing fast sorts target large arrays. Small arrays (< 200 elements) are called millions of times per second in real workloads.

- ▶ **CPU potential wasted**

Modern CPUs can compare 8 floats simultaneously via AVX registers — but standard sort never uses this.

Why It Is Important

- ▶ **7× speedup over std::sort**

Benchmarked on real hardware — not theoretical.

- ▶ **Cache-friendly**

Sorting stays inside CPU registers — almost zero memory traffic during sort.

- ▶ **Branch-free hot path**

Fixed comparison network means no mispredicted branches — the CPU pipeline stays full.

- ▶ **Real use cases**

Image compression (8×8 blocks), KD-tree construction, physics simulations — all sort small float arrays repeatedly.

- ▶ **Portable**

Single header file works on x86 (AVX) and ARM (NEON) with the same logic.

Core Idea

Central Contribution & Parallelism Strategy

What Kind of Parallelism?

- ▶ Data parallelism — the same min/max operation applied to 8 floats simultaneously
- ▶ AVX register holds 8 floats; one instruction compares all 8 pairs at once
- ▶ NEON register holds 4 floats; same principle on ARM
- ▶ No task parallelism — single threaded, but maximally wide per instruction

Parallelization Strategy

- ▶ Decomposition: array split into vectors (chunks of 8 floats each)
- ▶ Each vector sorted independently with `simd_sort_1V` (no communication needed)
- ▶ Orchestration: sorted vectors merged using bitonic merge network
- ▶ For large arrays: tiled merge sort — each tile ≤ 192 floats sorted by SIMD, then merged

Data Locality & Cache Use

- ▶ Small sort (≤ 192 floats) fits entirely in L1 cache — zero cache misses during sort
- ▶ Load once from RAM $\rightarrow$ sort inside registers $\rightarrow$ store once back to RAM
- ▶ Tile size (192) chosen so working set stays in L1/L2 cache
- ▶ Merge phase uses temporary buffers — only necessary memory copies made

Bitonic Network — The Core

- ▶ Fixed sequence of compare-and-swap operations — no data-dependent branching
- ▶ Round 1: compare distance-1 pairs $\rightarrow$ Round 2: distance-3 $\rightarrow$... $\rightarrow$ Round 6: full sort
- ▶ Every round: shuffle $\rightarrow$ parallel min $\rightarrow$ parallel max $\rightarrow$ blend result
- ▶ Same network topology regardless of input values — predictable pipeline behaviour

Approach

How the Problem Is Solved Step by Step

STEP 1

Load

Read floats from RAM into SIMD registers (8 at a time). Partial vectors padded with $+\infty$ so they sort to the end and are discarded on store.

STEP 2

Sort 1V

`simd_sort_1V`: 6 rounds of shuffle $\rightarrow$ min/max $\rightarrow$ blend inside one register. Sorts 8 floats with ~18 SIMD instructions. Zero memory access.

STEP 3

Sort NV

`simd_sort_NV` builds recursively: `sort_4V = sort_2V(a,b) + sort_2V(c,d) + merge_4V`. Scales from 2 to 24 vectors (up to 192 floats).

STEP 4

Merge

`simd_merge_2V_sorted`: reverse one vector, min/max against the other $\rightarrow$ bitonic sequence $\rightarrow$ aftermerge rounds finish it. All in registers.

STEP 5

Store

Write sorted registers back to RAM. Partial last vector writes only the real elements — $+\infty$ padding discarded.

Call Hierarchy

```
simd_merge_sort(array, n)
├─ n ≤ 192
│   └─ simd_small_sort()
│       ├── load into registers
│       ├── simd_sort_NV()
│       │   ├── simd_sort_1V() [6 rounds]
│       │   ├── simd_sort_1V()
│       │   └─ simd_merge_NV_sorted()
│       └─ store to memory
└─ n > 192
    └─ merge_sort()
        ├── sort left (recurse / simd_small_sort)
        ├── sort right (recurse / simd_small_sort)
        └─ merge_arrays() [SIMD merge]
```

Gap / Opportunity

Limitations & Further Optimization Potential

1 Single-Threaded Only

The entire sort runs on one CPU core. All other cores sit idle. For large arrays this is the biggest missed opportunity — the tile sort phase is embarrassingly parallel.

HIGH

2 Branchy SIMD Merge

`merge_arrays()` contains an if/else that decides whether to load from left or right half each iteration. This breaks CPU branch prediction and stalls the pipeline unnecessarily.

HIGH

3 No Prefetching

When processing large arrays the CPU frequently stalls waiting for data from RAM. `_mm_prefetch` hints could load the next chunk into cache while sorting the current one — essentially free latency hiding.

MED

4 AVX-512 Not Used

Modern Intel server and desktop CPUs support AVX-512 (16 floats per register vs 8). Porting the bitonic network to AVX-512 would double throughput per instruction with identical algorithm logic.

MED

5 Hardcoded Tile Size

`simd_small_sort_max()` always returns 192 regardless of CPU. Optimal tile size should match L1 cache size — a smaller L1 cache will suffer unnecessary cache misses with the current fixed value.

LOW

Proposed Approach

What We Will Implement, Adapt & Evaluate

Implement OpenMP Threading

- ▶ Parallelize the tile-sort phase using `#pragma omp parallel` for
- ▶ Each thread independently calls `simd_small_sort` on its tile
- ▶ Tiles are fully independent — zero synchronization needed during sort phase
- ▶ Merge phase remains sequential as baseline
- ▶ Expected: near-linear speedup with core count on large arrays

Implement Branchless Merge

- ▶ Replace the if/else branch in `merge_arrays` with masked SIMD loads
- ▶ Use `_mm256_blendv_ps` to select from left or right without branching
- ▶ Eliminates branch misprediction stalls in the merge hot loop
- ▶ Keeps the rest of the SIMD merge logic identical
- ▶ Expected: measurable throughput improvement on large arrays

Evaluate AVX-512 Width

- ▶ Port `simd_sort_1V` to AVX-512 — 16 floats per register instead of 8
- ▶ Bitonic network rounds stay identical, just wider shuffle/blend masks
- ▶ `simd_small_sort_max` grows from 192 to 384 floats
- ▶ Benchmark on AVX-512 capable hardware vs current AVX baseline
- ▶ Expected: up to 2x throughput on supported CPUs

Preliminary Plan

Tools, Languages, Platforms & Implementation Roadmap

Tools & Platform

Language	C99 / C++17
SIMD	Intel AVX2 intrinsics (<code>_mm256_*</code>) + ARM NEON (<code>arm_neon.h</code>)
Threading	OpenMP 4.5 (<code>#pragma omp parallel for</code>)
Compiler	GCC / Clang with <code>-mavx2 -fopenmp -O3</code>
Platform	x86-64 Linux / Windows (AVX2 required)
Benchmark	<code>sokol_time.h</code> (already in repo) + custom harness
Baseline	<code>std::sort</code> vs original <code>simd_merge_sort</code> vs new version
Profiling	<code>perf stat</code> / Intel VTune for cache miss & IPC analysis

Implementation Roadmap

Week 1 Baseline & Benchmarking	Set up benchmark harness. Profile existing code with <code>perf</code> /VTune. Record throughput for array sizes 8 → 10M floats. Establish baseline numbers for <code>std::sort</code> and current <code>simd_merge_sort</code> .
Week 2 OpenMP Tile Parallelism	Add <code>#pragma omp parallel for</code> over tile sort phase. Verify correctness with randomized test suite. Benchmark across 1, 2, 4, 8, 16 threads. Measure scaling efficiency.
Week 3 Branchless SIMD Merge	Replace <code>if/else</code> in <code>merge_arrays</code> with masked SIMD loads (<code>_mm256_blendv_ps</code>). Benchmark merge-dominated workloads (large arrays already sorted in tiles). Measure branch-miss reduction via <code>perf stat</code> .
Week 4 AVX-512 Port + Final Evaluation	Port <code>simd_sort_1V</code> and bitonic network to <code>__m512</code> (16-wide). Run full benchmark suite. Compile results table: <code>std::sort</code> vs original vs +OpenMP vs +branchless vs +AVX-512.

Summary

Problem

Small float arrays are sorted millions of times per second in real workloads — `std::sort` wastes CPU potential

Core Idea

Bitonic sort mapped onto SIMD registers — 8 comparisons per instruction, all work in registers, near-zero memory traffic

Approach

Fixed comparison network (shuffle → min → max → blend) × 6 rounds per vector, scaled up to 24 vectors via recursive merge

Gap

Single-threaded, branchy merge, no prefetch, no AVX-512, hardcoded tile size

Proposed

OpenMP tile parallelism + branchless merge + AVX-512 port — targeting measurable speedup over the already-fast baseline

Thank You